Feature Engineering & Data Preprocessing

What is Feature Engineering?

The art of turning raw data into signals a model can learn from.

Raw data rarely comes in the perfect shape for a machine learning model. Feature engineering is the process of using domain knowledge to create, transform, or select the input variables (features) that best represent the underlying problem — so the model can learn more easily and accurately.

HOW DO WE GET RAW DATA FOR MACHINE LEARNING?

Machine Learning models require data to learn patterns and make predictions. The first step in any ML project is collecting raw data from different sources.

1. User-Generated Data

- Online registration forms
- Job application forms
- Surveys and questionnaires
- Customer feedback forms
- Product reviews and ratings
- Social media interactions

Example: Netflix collects movie ratings and watch history from users.

---

2. Application Usage Data Data generated when users interact with applications.

- Clicks and page visits
- Search history
- Time spent on pages
- Purchase history
- Login activity
- User behavior tracking

Example: Amazon records products viewed, searched, and purchased.

---

3. Sensors and IoT Devices Physical devices continuously generate data.

- Smart watches
- Fitness trackers
- GPS devices
- Temperature sensors
- Traffic cameras

Example: A fitness tracker records heart rate and step count data.

---

4. Public Datasets Many organizations provide datasets for research and learning.

Sources:

- Kaggle
- UCI Machine Learning Repository
- Google Dataset Search
- Government Open Data Portals

Example: The Iris Flower Dataset is widely used for classification tasks.

---

5. Company Databases Organizations store large amounts of historical business data.

- Customer records
- Sales transactions
- Inventory data
- Banking records
- Healthcare records

Example: Banks use historical loan records to predict loan defaults.

---

6. APIs (Application Programming Interfaces) APIs provide structured access to data.

- Weather APIs

- Stock Market APIs
- News APIs
- Social Media APIs
- Sports APIs

Example: A weather prediction application collects temperature data through weather APIs.

---

7. Web Scraping Data can be collected from websites using scraping tools.

- Product prices
- News articles
- Hotel reviews
- Job postings
- Real estate listings

Example: House price data can be collected from real estate websites for price prediction models.

---

8. Beta Testing and Pilot Programs Before an application is fully launched, selected users test it.

Data collected:

- Feature usage
- Bugs and errors
- User feedback
- Performance metrics

Example: A new mobile application collects feedback from beta users before public release.

MACHINE LEARNING DATA PIPELINE

Data Sources

↓

Raw Data Collection

↓

Data Cleaning

↓

Feature Engineering

↓

Training Dataset

↓

Machine Learning Model

Real-world example: date of birth

A raw date-of-birth column is nearly useless as-is. Here's how to extract useful features from it:

Raw column	Extracted features	Why useful?
date_of_birth	age (in years)	Direct numeric signal
date_of_birth	generation (Boomer / Gen-Z)	Captures cohort behaviour
date_of_birth	birth_month (1-12)	Seasonal patterns
date_of_birth	birth_weekday (Mon-Sun)	Day-of-week patterns

feature extraction from dates

```
import pandas as pd
# Sample data
df = pd.DataFrame({'dob': ['1995-03-15', '1988-11-02', '2001-07-20']})
# Convert string to datetime first
df['dob'] = pd.to_datetime(df['dob'])
# Extract useful features
df['age'] = 2024 - df['dob'].dt.year
df['birth_month'] = df['dob'].dt.month
df['birth_day'] = df['dob'].dt.day_name() # e.g. 'Monday'
# Bin age into generations
bins = [0, 28, 44, 60, 100]
labels = ['Gen-Z', 'Millennial', 'Gen-X', 'Boomer']
df['generation'] = pd.cut(df['age'], bins=bins, labels=labels)
print(df[['age', 'birth_month', 'birth_day', 'generation']])
```

```
   age  birth_month  birth_day  generation
0   29            3  Wednesday  Millennial
1   36           11  Wednesday  Millennial
2   23            7     Friday      Gen-Z
```

Other common feature engineering tricks

Feature Engineering Technique	Example	Purpose / Benefit
Combine Columns (Ratio Features)	`price_per_sqft = price / area`	Creates a new feature that often carries more useful information than either col
Log Transformation	`log(salary)`	Reduces the impact of extreme outliers and makes skewed data more normally
Interaction Terms	`is_weekend AND is_sale`	Captures combined effects between features that individual columns may not re
Binning / Discretisation	`age: 0–18 → Minor`, `19–65 → Adult`, `65+ → Senior`	Converts continuous values into categories, making patterns easier for some mc
Polynomial Features	`x, x², x³`	Allows linear models to learn non-linear (curved) relationships in the data.

Handling Categorical Data

Models understand numbers, not words — so we have to translate. Most real-world datasets contain text categories: city names, product types, yes/no flags. ML algorithms can't do maths on the word 'Mumbai', so we must encode categories as numbers first.

Method 1 — Label Encoding

Replaces each category with an integer. Simple and compact, but it implies an ordering (2 > 1

0), which is only correct when the category genuinely has an order (e.g. Small < Medium < Large).

Original value	Encoded value
Yes	1
No	0
Small	0
Medium	1
Large	2

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
df['purchased'] = ['Yes', 'No', 'Yes']
df['size'] = ['Small', 'Medium', 'Large']
le = LabelEncoder()
df['purchased_enc'] = le.fit_transform(df['purchased'])
size_order = {'Small': 0, 'Medium': 1, 'Large': 2}
df['size_enc'] = df['size'].map(size_order)
print(df[['purchased', 'purchased_enc', 'size', 'size_enc']])
```

```
  purchased  purchased_enc    size  size_enc
0       Yes              1   Small         0
1        No              0  Medium         1
2       Yes              1   Large         2
```

Method 2 — One-Hot Encoding

Creates one new binary (0/1) column per category. No false ordering is implied. This is the standard approach for nominal categories (no natural order).

City	Delhi	Mumbai	Chennai
Delhi	1	0	0
Mumbai	0	1	0

```
import pandas as pd
df = pd.DataFrame({'city': ['Delhi', 'Mumbai', 'Chennai', 'Delhi']})
df_encoded = pd.get_dummies(df, columns=['city'], dtype=int)
print(df_encoded)
print("-----------------------------------------------")
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded = ohe.fit_transform(df[['city']])
print(encoded)
```

```
   city_Chennai  city_Delhi  city_Mumbai
0             0           1            0
1             0           0            1
2             1           0            0
3             0           1            0
-----------------------------------------------
[[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
```

When to use which?

Situation	Use	Example
Categories have a natural order	Label Encoding	Small / Medium / Large
Binary yes/no column	Label Encoding	Purchased: Yes/No
No order, few categories (<10)	One-Hot Encoding	City: Delhi/Mumbai/Chennai
No order, many categories (10+)	Target Encoding	Postcode, product SKU

Handling Missing Data

Real-world data is always incomplete — here's how to deal with it. Missing values are one of the most common data quality problems. If you pass a DataFrame with NaN values directly into sklearn, it will throw an error. You must decide: fill the gap, or drop the row?

Why does data go missing?

MCAR

Missing Completely At Random A sensor randomly fails. Nothing systematic — safe to drop rows.

MAR

Missing At Random Older people skip income questions more than younger. Pattern exists but is explainable.

MNAR

Missing Not At Random High earners hide their salary. The very fact of missing-ness carries signal — hardest to handle.

Step 1 — Diagnose missing values

```
import pandas as pd
import numpy as np
# Create a sample DataFrame with missing values
df = pd.DataFrame({
    'feature_A': [1, 2, np.nan, 4, 5],
    'feature_B': ['X', 'Y', 'Z', np.nan, 'Y'],
    'feature_C': [10.5, np.nan, 12.3, 14.0, np.nan]
})
print('Missing values count per column:')
print(df.isnull().sum())
print('\n')
print('Percentage missing per column:')
print(df.isnull().mean() * 100)
# !pip install missingno
# import missingno as msno
# msno.matrix(df)
```

```
Missing values count per column:
feature_A    1
feature_B    1
feature_C    2
dtype: int64


Percentage missing per column:
feature_A    20.0
feature_B    20.0
feature_C    40.0
dtype: float64
```

Step 2 — Fill or drop?

Scenario	Recommended Action	Reason
< 5% of rows missing in a column	Drop those rows	Data loss is minimal and unlikely to affect model performance.

Scenario	Recommended Action	Reason
5–30% missing, Numeric Column	Fill with **Mean** (if normally distributed) or **Median** (if skewed)	Preserves most data while handling missing values appropriately.
5–30% missing, Categorical Column	Fill with **Mode** (most frequent value)	Maintains category consistency and avoids losing records.
> 30% missing in a column	Consider dropping the entire column	Too much missing data may make the feature unreliable.
MNAR (Missing Not At Random)	Create a binary indicator feature (e.g., `was_salary_missing`)	Missingness itself may contain valuable information for prediction.

SimpleImputer — filling missing values

```
from sklearn.impute import SimpleImputer
import numpy as np
numeric_cols = ['feature_A', 'feature_C']
num_imputer = SimpleImputer(strategy='median')
df[numeric_cols] = num_imputer.fit_transform(df[numeric_cols])
categorical_cols = ['feature_B']
cat_imputer = SimpleImputer(strategy='most_frequent')
df[categorical_cols] = cat_imputer.fit_transform(df[categorical_cols])
print('Total missing values after imputation:', df.isnull().sum().sum()) # Should print 0
print('\nDataFrame after imputation:')
print(df)
```

```
Total missing values after imputation: 0

DataFrame after imputation:
   feature_A feature_B  feature_C
0        1.0         X       10.5
1        2.0         Y       12.3
2        3.0         Z       12.3
3        4.0         Y       14.0
4        5.0         Y       12.3
```

Outlier handling

Outliers are extreme values that can distort model training. Two common detection methods:

```
# Method 1: IQR (interquartile range) — robust to distribution shape
# Using 'feature_A' as a numerical column for demonstration
Q1 = df['feature_A'].quantile(0.25)
Q3 = df['feature_A'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
# Keep only rows within bounds
df_clean_iqr = df[(df['feature_A'] >= lower) & (df['feature_A'] <= upper)]
print("DataFrame after IQR outlier removal on 'feature_A':")
print(df_clean_iqr)
print('\n')
# Method 2: Z-score — works well for normally distributed data
from scipy import stats
# Using 'feature_A' for demonst ration
z_scores = stats.zscore(df['feature_A'])
df_clean_zscore = df[abs(z_scores) < 3] # remove rows where |z| > 3
print("DataFrame after Z-score outlier removal on 'feature_A' (threshold 3):")
print(df_clean_zscore)
```

```
DataFrame after IQR outlier removal on 'feature_A':
   feature_A feature_B  feature_C
0        1.0         X       10.5
1        2.0         Y       12.3
2        3.0         Z       12.3
3        4.0         W       14.0
4        5.0         W       12.3


DataFrame after Z-score outlier removal on 'feature_A' (threshold 3):
   feature_A feature_B  feature_C
0        1.0         X       10.5
1        2.0         Y       12.3
2        3.0         Z       12.3
3        4.0         W       14.0
4        5.0         W       12.3
```

Mean vs median

for imputation Use mean only when your data is roughly symmetric (no big outliers). Use median when data is skewed (income, house prices, etc.) — the median is not pulled by extreme values the way the mean is.

sklearn Pipelines

Chain every preprocessing step into one neat, leak-proof object. A Pipeline bundles preprocessing steps and a final estimator into a single object. You call .fit() once and everything runs in sequence — cleaning, scaling, then training. This prevents data leakage and makes deployment far simpler.

What is data leakage?

Data leakage — the silent model killer

Leakage happens when information from the test set 'leaks' into your training. Example: if you calculate mean salary on the WHOLE dataset before splitting, your test rows have already influenced the imputer. The model looks great in evaluation but fails in production. Pipelines prevent this by fitting only on training data and applying the same transform to test data.

Building a 3-step pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
# Assuming 'df' is the DataFrame from previous steps
# For demonstration, let's define X and y from the existing 'df'
X = df[['feature_A', 'feature_C']] # Using numeric features as X
# Create a dummy target variable 'y' for classification example
# e.g., if feature_A is greater than its median, classify as 1, else 0
y = (df['feature_A'] > df['feature_A'].median()).astype(int)
# Step 1: split data BEFORE building the pipeline
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Step 2: define the pipeline (a list of (name, transformer) tuples)
pipe = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # fill NaNs
    ('scaler', StandardScaler()), # scale features
    ('model', LogisticRegression(random_state=42)), # train classifier
])
```